SOFTWARE ENGINEERING

Project Title: LLM EVALUATION

& BENCHMARKING PLATFORM

DIGITAL ASSIGNMENT – 1

Team Members

Monish D
24BDS1033

Soaham S
24BDS1021

Abstract

Large Language Models (LLMs) such as GPT, Gemini, Claude, Llama, and Mistral are increasingly being used in software applications for question answering, content generation, coding, summarization, reasoning, and information retrieval. However, selecting an appropriate LLM for a particular application is challenging because different models exhibit different levels of accuracy, reasoning ability, reliability, latency, token efficiency, and cost.

The proposed project, **LLM Evaluation and Benchmarking Platform**, is a software platform designed to systematically evaluate and compare multiple Large Language Models using standardized datasets, prompts, evaluation methods, and performance parameters. The platform will allow users to select multiple LLMs, execute identical evaluation tasks across them, analyze their responses, and obtain a comprehensive comparative benchmark.

The system will evaluate multiple dimensions of LLM performance, including **accuracy, relevance, completeness, instruction following, reasoning ability, hallucination rate, factuality, groundedness, consistency, latency, token usage, and estimated cost**. Different evaluation techniques can be used depending on the task, including exact-match evaluation, statistical metrics, semantic similarity, reference-based evaluation, and LLM-as-a-Judge methods.

A major focus of the proposed platform is **hallucination detection**. The system will attempt to identify responses containing fabricated, factually incorrect, or unsupported claims. This enables the platform to evaluate not only how well an LLM answers a question, but also how reliably it produces trustworthy information.

The platform will provide comparative dashboards, performance metrics, visualizations, and benchmark reports. Instead of identifying a single universally "best" LLM, the system will help users determine which model is most suitable for a particular task based on their required balance of quality, reliability, speed, and cost.

1 Introduction

1.1 Background

Large Language Models have become a major component of modern artificial intelligence and software systems. Models such as GPT, Gemini, Claude, Llama, and Mistral are capable of performing a wide variety of tasks, including natural language generation, question answering, summarization, code generation, translation, reasoning, and information extraction.

Despite their broad capabilities, LLMs do not perform equally across all tasks. A model that performs well on mathematical reasoning may not necessarily be the best choice for summarization or coding. Similarly, a highly capable model may have higher API costs or greater response latency than a smaller model.

Another important problem associated with LLMs is **hallucination**. An LLM may generate information that appears convincing but is factually incorrect, fabricated, or unsupported by the available context. This is particularly important in applications involving education, healthcare, finance, research, customer support, and other domains where incorrect information can have significant consequences.

Traditional benchmarks provide useful information about general model capabilities, but developers may require evaluation using their own datasets, prompts, and application requirements. Manually testing several models using the same prompts is time-consuming and makes systematic comparison difficult.

The proposed LLM Evaluation and Benchmarking Platform provides a centralized environment for conducting controlled and reproducible evaluations of multiple LLMs.

1.2 Statement

The rapid growth in the number of proprietary and open-source LLMs has made model selection increasingly difficult. Existing benchmark scores provide general information about model performance, but they may not accurately represent how a model will perform for a specific application or dataset.

Manual evaluation requires a user to send identical prompts to different models, collect responses, compare the outputs, identify hallucinations, measure response time, monitor token usage, estimate costs, and calculate performance scores. Performing these activities repeatedly is inefficient and can result in inconsistent evaluation.

Furthermore, evaluating only accuracy is insufficient for modern LLM applications. A model may produce a relevant answer but contain fabricated facts, or it may provide a highly accurate answer while requiring significantly more time and cost than another model.

Therefore, there is a need for a unified platform capable of evaluating LLMs across multiple dimensions, including **quality, reliability, hallucination, efficiency, and cost**.

The proposed system addresses this problem by providing an automated benchmarking environment in which multiple LLMs can be evaluated using the same datasets, prompts, configurations, and evaluation criteria.

1.3 Motivation

The motivation for developing the LLM Evaluation and Benchmarking Platform comes from the increasing use of LLMs in real-world software systems and the difficulty of determining which model is most appropriate for a particular application.

The major motivating factors are:

1. **Rapid growth of LLMs:** New proprietary and open-source models are continuously being developed.
2. **Different model capabilities:** LLMs can perform differently across coding, reasoning, summarization, question answering, and other tasks.

3. **Hallucination problem:** LLMs can generate plausible but false or unsupported information.
4. **Need for reliable AI:** Applications increasingly require models that provide factually reliable and grounded responses.
5. **Cost differences:** Different models have different pricing structures and token costs.
6. **Latency requirements:** Real-time applications require models capable of producing responses quickly.
7. **Application-specific evaluation:** Developers may need to evaluate models using their own datasets and prompts rather than relying only on public benchmarks.
8. **Reproducibility:** Using identical inputs and configurations allows more meaningful comparisons between models.
9. **Automated evaluation:** Automating the process reduces the time and effort required to conduct large-scale model comparisons.
10. **Better model selection:** The platform can help users select a model based on their specific priorities rather than relying on a single overall benchmark score.

The project therefore aims to move from simple model comparison toward **multi-dimensional, task-specific, and reliability-aware LLM evaluation**.

1.4 Challenges

The development of an LLM benchmarking platform introduces several technical and software engineering challenges.

Model Integration

Different LLM providers use different APIs, authentication methods, request formats, response structures, model parameters, and pricing systems. A common abstraction layer is required to integrate these models into a single platform.

Evaluation Consistency

All selected models must receive equivalent prompts, datasets, and evaluation conditions. Differences in prompts or configuration could make the benchmark unfair.

Hallucination Detection

Identifying hallucinations is more difficult than calculating simple accuracy. Some responses contain both correct and incorrect claims. The system may therefore need to evaluate individual claims against reference information or source documents.

Open-Ended Response Evaluation

Many LLM tasks have multiple valid answers. Exact string matching is therefore insufficient. Semantic similarity, reference-based evaluation, or model-based judging may be required.

LLM-as-a-Judge Reliability

Using another LLM to evaluate responses can introduce evaluator bias. The platform must record the evaluation model and methodology and, where possible, combine multiple evaluation methods.

Latency Measurement

Response time can be affected by network conditions, server load, and provider infrastructure. The system must measure latency consistently and clearly define what is being measured.

Token and Cost Tracking

Different providers have different tokenization methods and pricing structures. The platform must collect token information where available and calculate estimated costs using the appropriate pricing configuration.

Reproducibility

LLM responses can change between executions due to temperature, model updates, system prompts, and other parameters. The platform should store relevant configurations for every benchmark execution.

Scalability

Large benchmark datasets executed against many models can generate a large number of API requests. The system must support batch processing and efficient job management.

API Failures and Rate Limits

External LLM APIs can experience network failures, rate limits, service interruptions, and authentication errors. The platform must handle these conditions without corrupting benchmark results.

2 Planning & Requirements Specification

2.1 System Planning

The development of the LLM Evaluation and Benchmarking Platform will follow a structured software engineering process. The project will be divided into multiple phases to ensure systematic development and testing.

Phase 1 – Requirement Analysis

The functional and non-functional requirements of the platform will be identified. This includes supported LLM providers, evaluation tasks, benchmark datasets, evaluation metrics, user interactions, and reporting requirements.

Phase 2 – System Design

The architecture of the system will be designed, including the frontend, backend, model integration layer, evaluation engine, database, and reporting components.

Phase 3 – LLM Integration

Interfaces will be developed for connecting different LLM providers. A common interface will allow the evaluation engine to communicate with different models using a standardized request format.

Phase 4 – Dataset and Benchmark Management

The system will support predefined evaluation datasets and, where appropriate, user-provided datasets. Each benchmark will contain prompts, expected answers or reference information, and task-specific evaluation criteria.

Phase 5 – Evaluation Engine Development

The evaluation engine will execute identical test cases against selected models and collect the generated responses. It will calculate quality, reliability, hallucination, efficiency, and cost-related metrics.

Phase 6 – Hallucination Evaluation

A dedicated evaluation component will analyze model responses for unsupported or factually incorrect claims. Depending on the benchmark, this can use reference answers, source documents, knowledge bases, semantic comparison, or an LLM-based evaluator.

Phase 7 – Benchmark and Report Generation

The collected results will be aggregated and presented using tables, charts, model rankings, and comparative reports.

Phase 8 – Testing

The system will undergo functional, integration, performance, security, and reliability testing.

Phase 9 – Performance Analysis

Multiple LLMs and benchmark datasets will be evaluated to demonstrate the usefulness of the platform and identify differences in model performance.

The development process can follow an iterative approach, allowing individual components to be implemented and tested independently.

2.2 Requirements

The requirements of the proposed system are divided into functional user requirements and non-functional requirements.

2.2.1 User Requirements

The platform should provide the following functionality:

1. Users should be able to access the LLM benchmarking platform through a web interface.
2. Users should be able to select one or more LLMs for evaluation.
3. Users should be able to select an existing benchmark dataset.
4. Users should be able to upload or configure custom evaluation datasets where supported.
5. Users should be able to define or select evaluation prompts.
6. Users should be able to configure model parameters such as temperature and maximum output tokens.
7. The system should execute the same evaluation inputs across the selected models.
8. The system should collect and store responses generated by each model.
9. The system should calculate appropriate evaluation metrics for each task.
10. The system should measure response latency.
11. The system should record input and output token usage where supported.
12. The system should calculate estimated model execution costs where pricing information is available.
13. The system should evaluate hallucination and factual reliability where a suitable reference or evaluation method is available.
14. The system should allow users to compare multiple models.
15. The system should display model performance through tables, charts, and other visualizations.
16. Users should be able to inspect individual model responses.
17. Users should be able to view the reasons or evaluation scores associated with a response where supported by the evaluation method.
18. Users should be able to view task-wise and overall model performance.
19. The system should maintain benchmark execution history.
20. Users should be able to generate or export benchmark reports.
21. The system should handle invalid inputs and API failures appropriately.
22. The system should store relevant benchmark configuration information for reproducibility.

Proposed Evaluation Parameters

The platform will evaluate LLMs across five major dimensions.

A. Quality

Parameter	Purpose
Accuracy	Measures whether the generated answer is correct
Relevance	Measures how closely the response addresses the given question
Completeness	Measures whether important information is included
Instruction Following	Measures whether the model follows the given instructions
Response Quality	Measures the overall quality and usefulness of the response

B. Reliability

Parameter	Purpose
Hallucination Rate	Measures the frequency of fabricated, incorrect, or unsupported claims
Factuality	Measures whether claims in the response are factually correct
Groundedness	Measures whether the response is supported by the provided context or source material
Consistency	Measures whether the model produces stable results across repeated evaluations
Citation Accuracy	Measures whether generated citations correctly support the associated claims

C. Capability

Parameter	Purpose
Reasoning Ability	Evaluates the model's ability to solve reasoning-based problems
Coding Performance	Evaluates code generation and programming-related tasks
Mathematical Performance	Evaluates mathematical problem-solving ability
Task Success Rate	Measures the percentage of evaluation tasks successfully completed

D. Efficiency

Parameter	Purpose
Latency	Measures the time required to generate a response
Input Tokens	Measures the number of tokens supplied to the model
Output Tokens	Measures the number of generated tokens
Throughput	Measures the number of tasks or tokens processed within a given time

E. Economics

Parameter	Purpose
Cost per Request	Estimates the cost of executing an individual request
Token Cost	Measures the cost associated with input and output tokens
Cost per Successful Task	Measures the estimated cost required to successfully complete a task

Hallucination Evaluation

Hallucination will be treated as a dedicated reliability metric rather than simply considering it part of accuracy.

A basic hallucination rate can be calculated as:

$$
\text{Hallucination Rate} = \frac{\text{Responses containing hallucinated claims}}{\text{Total evaluated responses}} \times 100
$$

A more detailed claim-level approach can be implemented for suitable datasets:

$$
\text{Claim Hallucination Rate} = \frac{\text{Unsupported or false claims}}{\text{Total evaluated claims}} \times 100
$$

This allows the system to distinguish between a response containing one incorrect claim and a response where most of the information is fabricated.

For example, if a response contains five factual claims and one of them is unsupported, the system can record a claim-level hallucination rate of 20%.

2.2.2 Non-Functional Requirements

Performance

The platform should process evaluation requests efficiently and should support batch evaluation for larger datasets.

Scalability

The system should allow additional models, providers, datasets, and evaluation metrics to be added without major modifications to the core application.

Reliability

The platform should handle API failures, network errors, rate limits, and incomplete requests without losing previously stored benchmark results.

Usability

The user interface should allow users to configure benchmarks, execute evaluations, and interpret results without requiring extensive technical knowledge.

Security

API keys and other authentication credentials must be securely stored and must not be exposed through the frontend or benchmark reports.

Maintainability

The system should use modular components so that individual model adapters, evaluation metrics, and UI components can be modified independently.

Extensibility

The architecture should support the future addition of new LLM providers, evaluation methods, datasets, and performance metrics.

Reproducibility

Each benchmark execution should store important configuration information such as:

- Model name
- Model version
- Prompt
- Dataset
- System instructions
- Temperature
- Maximum output tokens
- Evaluation method
- Evaluation model, if applicable
- Execution timestamp

Evaluation Accuracy

The system should clearly distinguish between objective evaluation metrics and subjective/model-based evaluation methods.

Fault Tolerance

Failures from individual model providers should not cause the entire benchmark to fail. The system should record failed requests separately and continue evaluation where possible.

2.3 System Requirements

2.3.1 Hardware Requirements

The proposed platform can operate on a standard development computer when cloud-based LLM APIs are used.

Minimum Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent
- **RAM:** 8 GB
- **Storage:** 20 GB available storage
- **Internet:** Stable broadband internet connection
- **Display:** 1366 × 768 resolution or higher

Recommended Hardware Requirements

- **Processor:** Intel Core i7 / AMD Ryzen 7 or equivalent
- **RAM:** 16 GB or higher
- **Storage:** 50 GB SSD or higher
- **Internet:** High-speed and stable internet connection

If the system is extended to evaluate locally hosted open-source LLMs, additional GPU resources may be required depending on the size and architecture of the models.

For example, smaller quantized models can potentially be evaluated on consumer hardware, while larger models may require dedicated GPUs or cloud computing resources.

2.3.2 Software Requirements

Operating System

The platform can be developed and deployed on:

- Windows
- Linux
- macOS

Programming Language

- **Python** for backend development and evaluation logic
- **JavaScript / TypeScript** for frontend development

Frontend

- **React.js**
- HTML

- CSS
- JavaScript / TypeScript
- Charting library such as Chart.js, Recharts, or equivalent

Backend

- **FastAPI** or Flask
- REST API architecture
- Python-based evaluation services

Database

- **PostgreSQL** for the full application
- SQLite can be used during initial development and testing

LLM Integration

The platform can be designed to support multiple providers, including:

- OpenAI models
- Google Gemini models
- Anthropic Claude models
- Meta Llama models
- Mistral models
- Other compatible hosted or open-source models

A provider abstraction layer will allow these models to be accessed through a common interface.

Evaluation Technologies

Depending on the benchmark and task type, the system can use:

- Exact Match
- Accuracy
- Precision
- Recall
- F1 Score
- BLEU
- ROUGE
- Semantic Similarity
- Embedding-based evaluation
- Reference-based factuality evaluation
- Groundedness evaluation
- LLM-as-a-Judge
- Custom task-specific evaluation metrics

Development and Testing Tools

- Visual Studio Code
- Git
- GitHub
- Postman
- Python virtual environment
- Web browser
- Docker, if containerization is required

3 System Design

The proposed LLM Evaluation and Benchmarking Platform will use a modular architecture consisting of a **frontend, backend, benchmark manager, LLM integration layer, evaluation engine, database, and reporting system**.

3.1 System Architecture

The major components of the proposed system are described below.

1. User Interface

The frontend will provide an interface through which users can:

- Select LLMs
- Select or upload datasets
- Configure prompts
- Configure model parameters
- Select evaluation metrics
- Start benchmark runs
- Monitor evaluation progress
- View individual responses
- Compare model performance
- View hallucination analysis
- View cost and latency information
- Generate benchmark reports

2. Backend Server

The backend will act as the central controller of the platform.

It will:

- Receive requests from the frontend
- Validate user configurations
- Manage benchmark jobs
- Communicate with the LLM integration layer
- Store evaluation results
- Calculate aggregate statistics
- Return results to the frontend

3. Benchmark Manager

The Benchmark Manager will control the execution of benchmark experiments.

It will manage:

- Dataset selection
- Prompt configuration
- Model selection
- Evaluation configuration
- Benchmark execution
- Test-case tracking
- Evaluation status

This component ensures that the same benchmark conditions are applied consistently to all selected models.

4. LLM Integration Layer

Different LLM providers expose different APIs. To solve this problem, the system will use provider-specific adapters behind a common interface.

For example:

Standard Evaluation Request → Provider Adapter → LLM API → Standardized Response

This approach allows the system to add new providers without changing the core evaluation engine.

5. Evaluation Engine

The Evaluation Engine is one of the primary components of the system.

It will:

1. Load the selected benchmark dataset.
2. Prepare standardized prompts.
3. Send the same test cases to the selected LLMs.
4. Collect generated responses.
5. Record latency.
6. Record token usage.
7. Calculate quality metrics.
8. Perform hallucination and factuality evaluation.
9. Calculate efficiency and cost metrics.
10. Aggregate the results.
11. Generate comparative scores.

6. Hallucination Detection Module

The Hallucination Detection Module will specifically evaluate whether generated responses contain unsupported or incorrect information.

Depending on the benchmark, it can use:

- Reference answers
- Ground-truth datasets
- Source documents
- Retrieval-based verification
- Semantic similarity
- Claim extraction
- LLM-as-a-Judge
- Rule-based checks for structured tasks

A potential workflow is:

Model Response → Claim Extraction → Claim Verification → Supported/Unsupported Classification → Hallucination Score

This module will allow the platform to identify models that produce fluent but unreliable answers.

7. Database

The database will store:

- Model information
- Provider information
- Benchmark datasets
- Evaluation tasks
- Prompts
- Model configurations
- Generated responses
- Evaluation scores
- Hallucination results
- Token usage
- Latency
- Cost
- Benchmark execution history

8. Reporting and Visualization Module

The Reporting Module will transform raw benchmark results into understandable information.

Possible visualizations include:

- Overall model comparison
- Accuracy comparison
- Hallucination rate comparison

- Factuality comparison
- Reasoning performance
- Latency comparison
- Token consumption
- Cost comparison
- Cost versus performance
- Task-wise performance
- Model ranking

The platform should avoid relying exclusively on a single overall score because different applications may prioritize different characteristics.

For example:

Model A: Higher accuracy but higher cost

Model B: Lower hallucination rate but higher latency

Model C: Lower accuracy but significantly lower cost

The platform should allow users to understand these trade-offs rather than simply declaring one model as universally superior.

3.2 Basic System Flow

The overall workflow of the proposed system is:

User

↓

Web Interface

↓

Select Models + Dataset + Evaluation Parameters

↓

Backend Server

↓

Benchmark Manager

↓

LLM Integration Layer

↓

Multiple LLM APIs

↓

Response Collection

↓

Evaluation Engine

↓

Quality + Reliability + Hallucination + Efficiency + Cost Evaluation

↓

Database

↓

Analytics & Visualization

↓

Comparative Benchmark Report

3.3 Proposed Benchmark Output

A benchmark report can provide a consolidated view such as:

Model	Accuracy	Hallucination Rate	Latency	Tokens	Cost	Overall Performance
Model A	92%	4%	1.8s	850	$X	High
Model B	89%	2%	2.4s	720	$Y	High
Model C	85%	8%	1.1s	600	$Z	Medium

The actual values will be generated by the evaluation system and will depend on the selected benchmark.

This type of output allows users to make decisions based on multiple factors instead of relying on a single benchmark score.

3.4 Future Extensibility

The system can be extended in future versions to include:

- Custom benchmark creation

- Automated dataset generation
- Prompt comparison and optimization
- Repeated-run consistency testing
- Advanced claim-level hallucination detection
- Retrieval-Augmented Generation evaluation
- Model recommendation based on user-defined priorities
- Cost-performance optimization
- Automated benchmark scheduling
- Local open-source model evaluation
- Human evaluation alongside automated evaluation
- Statistical confidence analysis
- Version tracking for models and benchmarks

The final objective of the system is to provide a **reproducible, multi-dimensional, and reliability-aware environment for evaluating Large Language Models**, enabling developers and researchers to identify the most suitable model for a specific task based on quality, hallucination, efficiency, latency, and cost.